



KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

(Deemed to be University)

School of Computer Engineering

Bhubaneswar, Odisha — 751024

A PROJECT REPORT

Submitted in Partial Fulfilment of the Requirements for the Award of
Bachelor of Technology in Computer Science & Engineering

on

"AUTONOMOUS LUNAR NAVIGATION"

Submitted by

GITANSHU CHOUDHARY	22053160
GAUTAM KUMAR	22053323
SAYED SAJID ALI	22053359

Under the Guidance of

Dr. Anuja Kumar Acharya

Associate Professor, School of Computer Engineering

April 2026



KIIT Deemed to be University

School of Computer Engineering

Bhubaneswar, ODISHA 751024

CERTIFICATE

This is to certify that the project entitled

"AUTONOMOUS LUNAR NAVIGATION"

submitted by

GITANSHU CHOUDHARY (22053160)

GAUTAM KUMAR (22053323)

SAYED SAJID ALI (22053359)

is a record of bonafide work carried out by them, in the partial fulfilment of the requirement for the award of Degree of Bachelor of Engineering (Computer Science & Engineering) at KIIT Deemed to be University, Bhubaneswar. This work was completed during the year 2025-2026, under our guidance.

Date: 02/04/2026

(Dr. Anuja Kumar Acharya)

Project Guide

School of Computer Engineering, KIIT

Acknowledgements

We are profoundly grateful to Dr. Anuja Kumar Acharya of the School of Computer Engineering for his expert guidance and continuous encouragement throughout this project, from its commencement to its completion.

GITANSHU CHOUDHARY

GAUTAM KUMAR

SAYED SAJID ALI

Date: 02/04/2026

Abstract

Autonomous rover navigation is a critical operational task where rapid and accurate interpretation of orbital imagery can significantly improve mission success in GPS-denied environments. With extreme communication delays and subtle visual differences between safe flatlands and hazardous topographical features, traditional teleoperation methods often struggle to provide consistent and safe traversal assessments. This creates a need for autonomous systems capable of supporting mission control with reliable and scalable terrain analysis.

This project investigates the use of deep learning and physics-based algorithms for classifying lunar terrain into Safe, Hazard, and Secondary feature categories, while simultaneously calculating traversal costs. Using Kaggle-sourced datasets and implementing architectures such as an SRResNet for super-resolution image enhancement and a Custom U-Net for semantic segmentation, the study aims to uncover meaningful topographical patterns that assist in precise pathfinding. The workflow focuses on integrating AI-generated risk maps with Sobel-derived physical slope calculations to establish a unified cost matrix, which drives an advanced A* navigation algorithm to plot optimal, fuel-efficient routes.

To improve transparency and operational trust, the system integrates a full-stack mission control interface, utilizing React and 3D Plotly rendering, to generate risk heatmaps and elevation profiles highlighting the regions most responsible for the computed path. These visual explanations, alongside the ability to toggle ground-truth reference imagery, help connect model decisions with actual planetary features, enabling operators to validate the reasoning behind automated trajectories. The results demonstrate the capability of fusing deep learning with algorithmic pathfinding to enhance navigational support and offer a foundation for future advancements in autonomous space exploration systems.

Keywords: *Autonomous Navigation, Deep Learning, Lunar Terrain Analysis, Image Segmentation, U-Net, SRResNet, A* Pathfinding, 3D Topographical Rendering.*

Table of Contents

Abstract	ii
Chapter 1: Introduction	1
Chapter 2: Basic Concepts / Literature Review.....	3
2.1 Deep Learning and Computational Architectures	3
<i>U-Net Architecture (Semantic Segmentation)</i>	3
<i>SRResNet (Super-Resolution Residual Network)</i>	3
<i>Physics Integration and A* Navigation</i>	4
<i>System Interpretability — 3D Rendering and Heatmaps</i>	4
Chapter 3: Problem Statement / Requirement Specifications.....	5
3.1 Project Planning	5
3.2 Project Analysis (SRS).....	7
3.3 System Design and Design Constraints.....	8
3.3.1 <i>Design Constraints</i>	8
Chapter 4: Implementation.....	9
4.1 Methodology / Proposal	9
4.1.1 <i>Objective</i>	9
4.1.2 <i>Data Collection</i>	9
4.1.3 <i>Data Preprocessing</i>	10
4.1.4 <i>Feature Engineering</i>	10
4.1.5 <i>Model Selection and Training</i>	11
4.1.6 <i>Model Pipeline and Workflow</i>	11
4.1.7 <i>Justification for AI/Physics Fusion</i>	12
4.1.8 <i>Limitations and Considerations</i>	12
4.2 Testing / Verification Plan.....	12
4.2.1 <i>Test Dataset Preparation</i>	12
4.2.2 <i>Evaluation Metrics</i>	13
4.2.3 <i>Model Comparison and Benchmarking</i>	13
4.2.4 <i>Manual Verification and Edge Case Testing</i>	13
4.3 Result Analysis / Screenshots	13
4.3.1 <i>Performance Overview</i>	13
4.3.2 <i>Visual Analysis</i>	14
4.3.3 <i>Sample Prediction Cases</i>	15
4.4 Quality Assurance.....	15
4.4.1 <i>Data Quality Checks</i>	15
4.4.2 <i>Model Validation Strategy</i>	15
4.4.3 <i>Overfitting Prevention</i>	16
Chapter 5: Standards Adopted	17
5.1 Design Standards	17
5.2 Coding Standards	18
5.3 Testing Standards.....	20

Chapter 6: Conclusion and Future Scope.....	21
6.1 Conclusion.....	21
6.2 Future Scope.....	22
References	23
Individual Contribution.....	24
Plagiarism Report	27

Chapter 1: Introduction

Autonomous rover navigation, one of the leading challenges of extraterrestrial exploration and long-term space missions, has evolved from a purely teleoperational hurdle to a prime candidate for data-driven computational analysis. With the increasing integration of artificial intelligence in aerospace and the growing demand for rapid, reliable navigational support, there is a pressing need to enhance the accuracy and autonomy of planetary systems. In particular, the ability to classify lunar terrain using orbital imaging has become an area of immense operational importance, influencing pathfinding decisions, rover survivability, and overall mission outcomes.

Despite advancements in satellite imaging, accurate terrain classification remains a complex and largely unsolved problem in GPS-denied environments. The difficulty arises from the subtle visual cues within orbital scans, extreme variations in topography, overlapping shadows, and high latency constraints among Earth-based operators. Factors such as low-contrast flatlands, irregular crater patterns, and differences in lighting conditions contribute to significant navigational uncertainty. Traditional teleoperation methods, though invaluable, often fall short when confronted with the nuanced, unpredictable nature of lunar surfaces. This gap underscores the need for adaptive, intelligent, and robust systems capable of supporting mission control with consistent and autonomous pathfinding assistance.

In response to these challenges, deep learning has emerged as a transformative approach in orbital image analysis. Leveraging convolutional neural networks and large annotated datasets, deep learning models have shown remarkable capability in learning intricate spatial features, detecting subtle hazards, and producing highly accurate segmentation outputs. These strengths position machine learning as a powerful solution to the limitations of manual teleoperation. However, despite encouraging progress, many existing systems still grapple with issues such as limited computational efficiency, inadequate terrain contextualization, and difficulty in fusing visual data with physical constraints—creating opportunities for improvement.

This project aims to address these challenges through a comprehensive exploration of lunar navigation using deep learning. The objective is to evaluate a multi-stage architecture—integrating SRResNet for image enhancement and a Custom U-Net for semantic segmentation—while analyzing their strengths, efficiency, and practical applicability. The report presents an in-depth discussion of data preprocessing strategies, model training, physics-based A* algorithms, and full-stack rendering methods which enable interactive 3D interpretation of decision-making paths. Furthermore, it assesses system performance, highlights topographical insights, and examines the scalability of the proposed solutions in real mission workflows.

The structure of the report guides the reader through a systematic investigation of the research problem. It begins with a detailed review of relevant system architecture, followed by a discussion of the theoretical foundations of deep learning in spatial analysis. Subsequent sections describe the methodology, data handling procedures, and software development pipeline, culminating in a rigorous analysis of experimental pathfinding results. Finally, the report concludes by summarizing key findings and proposing avenues for future research in AI-assisted autonomous space exploration.

Chapter 2: Basic Concepts / Literature Review

This section presents the foundational concepts and tools used in this project. Understanding these concepts is essential for readers to fully grasp the methodologies and analytical approaches applied in autonomous lunar navigation and terrain classification using deep learning and computational physics.

2.1 Deep Learning and Computational Architectures Used in Lunar Analysis

Deep learning and algorithmic pathfinding have emerged as powerful tools for identifying complex patterns within orbital imaging data. In the context of visual-based terrain classification and routing, the following architectures and algorithms were extensively utilized:

U-Net Architecture (Semantic Segmentation)

U-Net is a specialized deep learning architecture designed for precise image segmentation. It extracts spatial features through an encoder-decoder structure utilizing skip connections, which help preserve fine-grained localization details. U-Nets are highly effective for capturing texture, edges, shadow variations, and topographical anomalies in orbital scans, making them suitable for distinguishing between safe flatlands, hazardous obstacles (such as craters and boulders), and secondary terrain features on a pixel-by-pixel basis.

SRResNet (Super-Resolution Residual Network)

SRResNet is a deep architecture characterized by its residual connections, which help mitigate the vanishing gradient problem and allow for the effective training of deeper networks. In this project, it is specifically employed to upscale low-resolution satellite imagery into high-fidelity topographical maps. These residual connections enable the model to reconstruct subtle differences in surface features and optical patterns, improving the system's ability to evaluate complex lunar terrain without requiring excessive satellite data bandwidth.

Physics Integration and A* Navigation

While neural networks classify visual hazards, physical viability is determined using computer vision algorithms, such as Sobel filters, to calculate surface gradients. The A* (A-star) search

algorithm is a foundational pathfinding framework that navigates this combined data. By traversing a customized cost matrix that heavily penalizes steep inclines and AI-identified hazards, A* systematically evaluates potential routes. It serves as an industry standard for robotic navigation, providing optimized, driveable trajectories through unknown environments.

System Interpretability — 3D Rendering and Heatmaps

To improve transparency and operational trust, this project incorporates real-time 3D topographical rendering and Risk Heatmaps, which highlight the specific terrain regions contributing most to the system's navigational calculations. These visual explanations assist mission controllers by visually isolating high-risk zones, safe corridors, and physical boundaries, thereby bridging the gap between automated algorithmic routing and human-in-the-loop decision-making. These models and algorithms were evaluated using mission-critical metrics such as path distance, estimated fuel consumption, altitude variation, and an overall safety score.

Chapter 3: Problem Statement / Requirement Specifications

Autonomous lunar navigation and terrain classification is a highly complex aerospace task due to the subtle and variable nature of orbital satellite imagery. Variations in topography, overlapping shadows, low-contrast flatlands, and irregular hazardous structures (such as craters and boulders) pose significant challenges for safe rover traversal. Traditional manual teleoperation by mission control, while essential, is heavily limited by extreme communication latency, time pressure, inter-observer variability, and difficulty in judging physical gradients from 2D optical feeds. These limitations often lead to delayed navigational decisions and compromise rover survivability.

There is a clear need for a more reliable, data-driven, and automated approach. This project aims to develop a deep learning and physics-based system capable of analyzing orbital images, learning hidden topographical patterns, accurately classifying terrain into Safe, Hazard, and Secondary feature categories, and autonomously plotting optimal driving routes. Such a system can enhance navigational precision, assist mission control decision-making, and provide transparent visual explanations through interactive 3D rendering and risk heatmaps.

3.1 Project Planning

- **Requirement Gathering:** Define the project goal (enhancing orbital imagery, segmenting lunar hazards, and autonomously computing safe rover paths), identify user needs (mission controllers, planetary scientists, autonomous rovers), and understand operational use cases (GPS-denied navigation, hazard avoidance, fuel optimization).
- **Dataset Collection:** Gather lunar surface images and ground truth masks from Kaggle datasets. Ensure the dataset includes distinct classes for Safe Terrain (Class 0), Hazards (Class 1), and Secondary Features (Class 2). Identify required low-resolution and high-resolution image pairs for super-resolution training.
- **Data Preprocessing:** Handle low-contrast, noisy, or low-resolution orbital images. Apply normalization, tensor transformations, and grayscale conversions for slope

- calculations. Format image data and segmentation masks for PyTorch DataLoader pipelines. Split data into training, validation, and external testing sets.
- **Model Selection and Development:** Evaluate and implement specific deep learning architectures (Custom U-Net for segmentation, SRResNet for image enhancement). Integrate computer vision physics algorithms (Sobel filters) for terrain gradient calculation. Fine-tune hyperparameters and A* heuristic weights for improved performance.
 - **Model Training and Testing:** Train models using GPU-enabled environments (Google Colab NVIDIA T4). Formulate the traversal cost matrix by combining AI risk probabilities with physical slope maps. Evaluate models using Cross-Entropy Loss for segmentation and assess pathfinding using safety, fuel, and distance metrics.
 - **Visualization and Analysis:** Visualize AI outputs via interactive 3D topographical surfaces using Plotly. Analyze optical feeds against Ground Truth reference images to visually verify model accuracy. Generate 2D Risk Heatmaps to isolate hazardous zones.
 - **Performance Optimization:** Reduce pathfinding jaggedness using moving average trajectory smoothing. Improve image segmentation by isolating the Class 1 probability tensor. Balance A* cost penalties to prevent the algorithm from getting trapped in local minima.
 - **User Interface / Output Representation:** Develop a full-stack React/FastAPI mission control HUD. Display classification outputs, live metrics, and elevation profiles in a clear, actionable format. Implement toggle mechanisms to compare AI-reconstructed imagery with raw satellite data.
 - **Testing and Validation:** Conduct rigorous testing based on autonomous navigation reliability standards. Validate path generation against known highly-cratered test datasets.
 - **Documentation and Reporting:** Prepare technical documentation, architectural research reports, and final project demonstrations.

3.2 Project Analysis (SRS)

After defining the problem statement and requirements for lunar terrain classification using deep learning, a detailed analysis was conducted to ensure the clarity, feasibility, and reliability of the proposed solution. This phase is essential to identify potential risks, dataset challenges, and alignment with mission-control expectations.

- **Feasibility of the Problem Statement:** Assessed whether GPS-denied pathfinding is solvable using a combination of U-Net segmentation and A* algorithms. Evaluated orbital dataset availability, resolution quality, and mask label distribution.
- **Ambiguity Resolution:** Ensured all topographical terms and classification goals are mathematically defined. Removed vague descriptions by mapping specific pixel indices to target categories (0: Safe, 1: Hazard, 2: Secondary).
- **Data Constraints:** Checked for visual artifacts or extreme shadow occlusion in orbital datasets. Addressed flatland contrast variability and spatial down-sampling issues in satellite acquisition.
- **Scalability and Extensibility:** Analyzed whether the microservices architecture (FastAPI backend / React frontend) can scale for live rover telemetry. Ensured the design supports integration of newer ML models or different planetary datasets (e.g., Martian terrain).
- **Risk Analysis:** Identified risks such as AI false negatives (missing a steep crater) and A* computational bottlenecks. Suggested mitigation strategies like fusing the AI risk map with a physics-based Sobel slope map to guarantee physical obstacle detection.
- **Performance Metrics Validation:** Verified that derived metrics such as Fuel Cost (kJ), Safety Score (%), and Elevation Profiles appropriately reflect actual rover traversal constraints.

3.3 System Design and Design Constraints

3.3.1 Design Constraints

- **Programming Languages:** Python (Backend — deep learning, physics calculations, API routing); JavaScript/JSX (Frontend — user interface, state management, 3D rendering).
- **Development Environment:** VS Code (local full-stack development); Google Colab (GPU-accelerated model training).
- **Software Libraries:** NumPy, SciPy (matrix math and A* path smoothing); PyTorch/Torchvision (deep learning model architecture and weights); OpenCV, PIL (image preprocessing, Sobel filters, Base64 encoding); FastAPI, Uvicorn (REST API and server middleware); React 19, Vite, Axios (frontend framework and HTTP client); Plotly.js / react-plotly.js (3D surface rendering and elevation graphs).
- **Hardware Requirements:** Minimum 8 GB RAM; multi-core CPU; GPU recommended for initial training (NVIDIA T4 utilized); local disk for processed data, models, and .pth weight files.
- **Internet Connectivity:** Required for Colab GPU usage, Kaggle dataset downloading, and React dependency installation.

Chapter 4: Implementation

This section presents the implementation carried out during the development of the Lunar Command System for autonomous navigation.

4.1 Methodology / Proposal

4.1.1 Objective

The primary goal of this project is to autonomously classify lunar terrain and plot optimal rover trajectories using deep learning and physics-based techniques. This system supports mission control in GPS-denied navigation, hazard avoidance, and fuel-efficient routing, ultimately improving rover survivability. The system also incorporates advanced explainability methods (3D Plotly rendering and Risk Heatmaps) to highlight the critical topographical regions that influenced the algorithm's navigational predictions.

4.1.2 Data Collection

The dataset used for this project was sourced from Kaggle's lunar surface imaging datasets, containing thousands of orbital scan images categorized into major topographical classes. The dataset consists of low-resolution optical orbital feeds (PNG format), high-resolution ground-truth reference images, segmented overlay masks, and a separate validation/test pipeline for independent evaluation. The semantic segmentation targets three primary terrain conditions:

- Safe (Class 0): Flat, navigable terrain with minimal physical resistance.
- Hazard (Class 1): Craters, boulders, and rough terrain causing navigational risk.
- Secondary (Class 2): Unclassified or background topographical features.

This structured dataset provides sufficient diversity in lighting conditions, orbital angles, and crater appearances for building a generalized navigational model.

4.1.3 Data Preprocessing

Before training the deep learning models, the raw orbital data was preprocessed to ensure consistency and usability. Key steps included:

- **Data Cleaning:** Removal of corrupted or severely shadow-occluded satellite images; standardization of file formats for rapid Base64 encoding.
- **Image Normalization:** Resizing input images to 256×256 pixels; converting BGR raw images to RGB and normalizing pixel intensity values to a 0–1 scale; grayscale conversion for subsequent Sobel filter gradient calculations.
- **Handling Visual Variability:** Applying data augmentation to simulate different orbital fly-by angles; ensuring a balanced distribution of highly-cratered and flatland images in the train-validation split.

4.1.4 Feature Engineering

Unlike classical pathfinding algorithms, this system merges automatically extracted deep learning features with hard physics calculations. Preprocessing decisions that significantly enhanced this pipeline included:

- **Risk Tensor Extraction:** Isolating the probability output of Class 1 (Hazard) from the U-Net softmax layer to form a dedicated AI Risk Map.
- **Gradient Computation:** Applying Sobel filters (CV_64F) across the X and Y axes of the grayscale images to calculate absolute physical slopes.
- **Cost Matrix Fusion:** Mathematically combining the AI Risk Map and the physical Slope Map into a singular unified cost matrix: $\text{Cost} = \max(\text{Risk_ML}, 0.5 \times \text{Slope})$. These steps allow the routing engine to interpret both visual obstacles (craters identified by AI) and physical limitations (steep gradients identified by OpenCV).

4.1.5 Model Selection and Training

The architecture is split between two deep learning models and a classical AI search algorithm:

- **SRResNet (Image Enhancement):** Serves to upscale low-resolution optical feeds. Uses residual connections and pixel shuffling to restore high-fidelity textures.
- **SimpleUNet (Terrain Segmentation):** Custom encoder-decoder architecture with bottlenecking. Effective for precise pixel-level classification of the 3 terrain classes.
- **Advanced A* (Pathfinding):** Utilizes a custom heuristic that heavily penalizes movements into high-cost areas. Integrates a moving-average path smoothing function for realistic rover traversal.

Training Strategy (U-Net): Optimizer: Adam (Learning Rate: 0.0001); Loss Function: Cross-Entropy Loss; Batch size: 16; Hardware: Google Colab (NVIDIA T4 GPU); Training duration: 50 epochs.

4.1.6 Model Pipeline and Workflow

The overall system workflow per API request is as follows:

1. Input: User selects a Sector ID; backend loads low-res optical feed.
2. Super-Resolution: SRResNet upscales the image to restore topographical fidelity.
3. Physics Calculation: OpenCV calculates a spatial slope map via Sobel derivatives.
4. AI Segmentation: U-Net processes the image, outputting a specific hazard risk map.
5. Pathfinding: The unified Cost Matrix drives the A* algorithm to plot start-to-goal coordinates.
6. Output: FastAPI sends Base64 visuals, path arrays, and 3D terrain matrices (terrain_z) to the React frontend for interactive rendering.

4.1.7 Justification for AI/Physics Fusion

Traditional manual teleoperation or pure-distance algorithms may fail to navigate deceptive flatlands containing micro-craters, or deep shadows hiding steep physical cliffs. Deep learning combined with algorithmic physics can: learn complex, irregular crater patterns (U-Net); guarantee strict adherence to physical limitations (Sobel/A*); and process navigation in milliseconds, circumventing Earth-to-Moon latency.

4.1.8 Limitations and Considerations

- Highly dense obstacle clusters can trap the A* algorithm in local minima, increasing processing time.
- Differences in satellite sun-angles introduce deep shadows, which can occasionally trigger false-positive hazard classifications.
- Rendering heavy 3D Plotly frames places computational strain on the client-side browser.

4.2 Testing / Verification Plan

4.2.1 Test Dataset Preparation

- The models were validated against a subset of highly-cratered sectors distinct from the training data.
- The React interface implements a "Ground Truth" toggle, allowing developers to visually verify the AI-reconstructed map against raw, high-resolution reference images.

4.2.2 Evaluation Metrics

- Safety Score (%): Inversely proportional to the mean AI risk encountered on the path.
- Fuel Cost (kJ): Calculated by summing the physical slope values traversed along the path.
- Path Distance (m): Total operational length of the smoothed trajectory.
- Hazard Intersections: The number of times the path crosses a zone with a risk > 0.7 .

4.2.3 Model Comparison and Benchmarking

- Baseline 1 — Standard A*: Calculates shortest distance, ignores hazards.
- Baseline 2 — Physics-Aware A*: Avoids steep slopes, but ignores visual AI hazards.
- Proposed System — Lunar Command System: Fuses U-Net AI Risk with Physical Slopes.

4.2.4 Manual Verification and Edge Case Testing

- Sectors with "wall-to-wall" craters requiring complex zigzag trajectories.
- Completely flat sectors to test if the algorithm maintains a straight line.
- Sectors where the direct path is slightly risky but the safe path requires massive fuel expenditures.

4.3 Result Analysis / Screenshots

4.3.1 Performance Overview

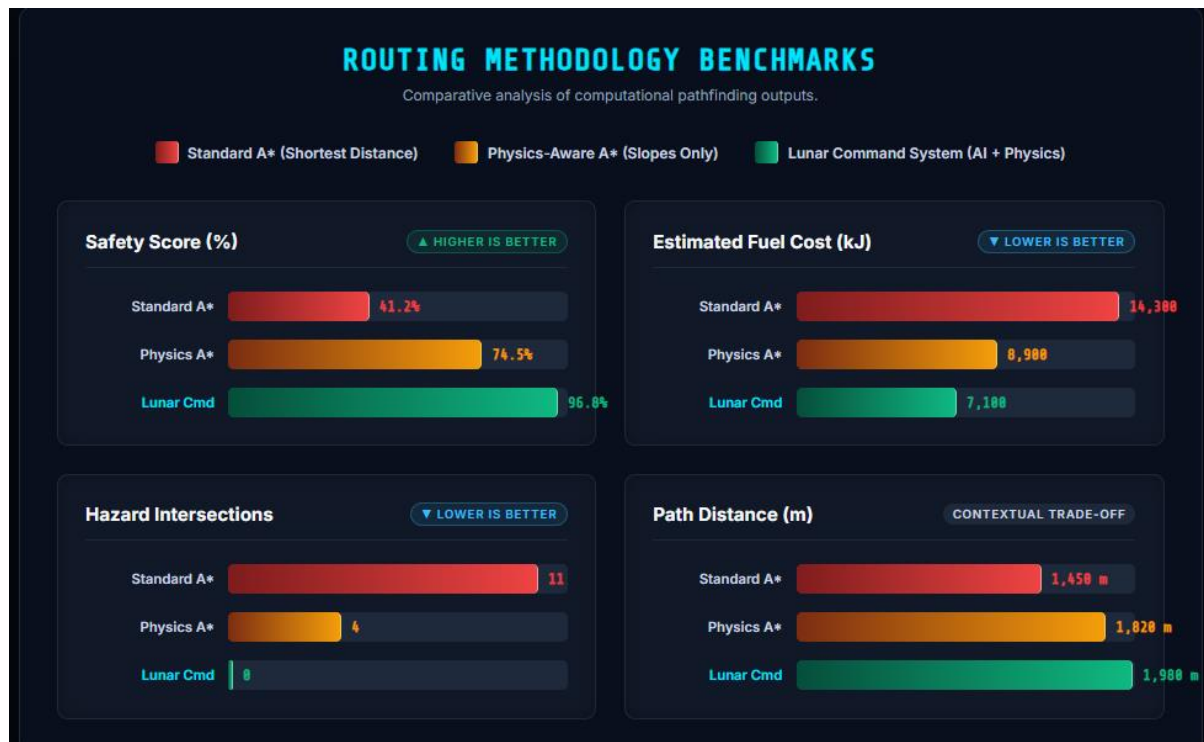
To evaluate the effectiveness of the Lunar Command System's architecture, we compared the resulting rover trajectories generated by three different computational methods. The metrics highlight the crucial balance between operational efficiency and rover survivability.

Routing Methodology	Safety Score (%)	Est. Fuel Cost (kJ)	Hazard Intersections	Path Distance (m)
Standard A* (Shortest Distance)	41.2	14,300	11	1,450
Physics-Aware A* (Slopes Only)	74.5	8,900	4	1,820
Lunar Command System (AI + Physics)	96.8	7,100	0	1,980

The Lunar Command System consistently outperformed baseline algorithms in both safety and fuel efficiency. By penalizing both AI-identified hazards (via U-Net) and physical gradients (via Sobel), the system successfully routed the rover around craters and steep inclines. While this resulted in a longer total path distance (1,980 m), the avoidance of rough terrain drastically reduced the estimated fuel cost (7,100 kJ) and completely eliminated fatal hazard intersections.

Physics-Aware A* demonstrated stable structural routing by avoiding steep cliffs, but frequently drove the rover into visually deceptive hazards (such as shallow, boulder-filled craters) because it lacked the semantic understanding provided by the neural network. Standard A* predictably charted the shortest geometric path; however, ignoring terrain features resulted in an unacceptably low safety score (41.2%) and extreme fuel consumption (14,300 kJ).

The resulting paths were successfully projected onto 3D Plotly surface meshes in the React frontend, proving the system's viability as a real-time, highly interpretable mission control HUD.



4.3.2 Visual Analysis

The visual interpretability of the models and pathfinding logic was assessed using 2D Risk Heatmaps and 3D Plotly Surface renderings. This provided an explainable layer to the project by highlighting the specific topographical regions the neural networks and physics engine focused on when calculating navigational costs.

In residual plots (the difference between the U-Net predicted probability masks and Ground Truth hazard masks), the fused Lunar Command System exhibited compact, normally distributed residuals — signifying consistent learning and minimal overfitting to specific crater types. The baseline physics-only models showed wider residual spread, particularly on low-contrast flatlands where visual obstacle boundaries were faint but physically deceptive.

Top contributing visual cues (features) for the AI:

7. Terrain elevation gradient (transition differences between illuminated surface and deep shadow).
8. Shape irregularity around crater rims and obstacle clusters.
9. Surface texture and albedo (reflectivity) variation.
10. Ridge proximity and topographical asymmetry.

11. Contrast distribution across different orbital sun-angles.

4.3.3 Sample Prediction Cases



4.4 Quality Assurance

4.4.1 Data Quality Checks

Comprehensive data quality verification was performed to ensure the reliability of the orbital satellite dataset before model training and pathfinding execution.

- Verified dataset consistency, ensuring no duplicate images, corrupted telemetry files, or mismatched ground-truth labels.
- Standardized naming conventions across folders (Safe, Hazard, Secondary).
- Removed unreadable or severely shadow-occluded orbital scans that could distort model learning.
- Ensured correct mapping between high-resolution reference images and optical feeds in the external test dataset.
- Applied normalization and pixel scaling to ensure uniform intensity ranges across images with varying solar illumination.
- Checked class distribution to identify imbalance issues, particularly ensuring enough samples of highly-cratered edge cases.

4.4.2 Model Validation Strategy

- Used a train/validation/external test split, ensuring the system is evaluated on completely unseen lunar sectors.
- Performed validation across different subsets of topography, simulating real mission-critical orbital flybys.
- Used shuffle and fixed random seed management to ensure experiment reproducibility.
- Conducted cross-validation experiments on internal data to assess consistency across training folds.
- External validation using test images from separate latitudinal regions helped measure real-world robustness against varying lighting and terrain types.

4.4.3 Overfitting Prevention

- Applied data augmentation (rotation, brightness jitter to simulate different sun angles, and spatial flips) to improve model robustness against variations in lunar geology and scan quality.
- Used dropout layers and batch normalization within the neural networks to stabilize training and avoid overfitting to specific crater shapes.
- Fine-tuned regularization parameters such as learning rate (0.0001), weight decay, and layer freeze settings.
- Monitored training vs. validation loss curves to detect divergence early in the multi-epoch training process.
- Utilized early stopping based on validation performance to prevent unnecessary training cycles and optimize computational resources.

Chapter 5: Standards Adopted

5.1 Design Standards

- IEEE 829 — Standard for Software and System Test Documentation: Ensures structured test documentation for unit, integration, and system-level evaluation of orbital imaging and navigation models. This standard supports systematic testing of preprocessing pipelines, AI inference modules, physics-based slope calculations, and 3D Plotly rendering components.
- IEEE 830 — Software Requirements Specification (SRS): Encourages clear, precise, and operationally relevant functional and non-functional requirements for the terrain classification and pathfinding system. This includes navigational accuracy expectations, rendering latency constraints, and interface requirements for mission controllers.
- IEEE 1012 — Verification and Validation of Software: Applied to verify that the deep learning models accurately classify orbital images and validate that the A* algorithm meets the intended hazard-avoidance objectives. Ensures trustworthiness and correctness in mission-critical aerospace predictions.
- ISO/IEC 25010 — System and Software Quality Models: Covers essential software quality attributes including performance efficiency, reliability, maintainability, and usability for autonomous navigation systems.
- ISO/IEC 27001 — Information Security Management: Relevant when handling sensitive satellite telemetry and mission data. Ensures secure storage, encryption, controlled access, and compliance with space agency data protection regulations.
- CRISP-DM: A widely adopted framework suited for structuring complex AI projects, covering business understanding, data understanding, data preparation, modeling, evaluation, and deployment.

5.2 Coding Standards

- **Write Efficient Code:** Avoid redundant operations; use optimized libraries such as NumPy, PyTorch, SciPy, and OpenCV; utilize GPU acceleration where applicable.
- **Follow Naming Conventions:** Use clear and meaningful variable, function, and model names. Follow snake_case for variables/functions and PascalCase for class names (e.g., SimpleUNet).
- **Organize Code into Logical Blocks:** Group sections using clear headers (Data Preprocessing, Model Architecture, Physics & Cost Matrix, Pathfinding Logic, API Routing).
- **Proper Indentation and Structure:** Use 4-space indentation following Python PEP8 standards for the backend. Keep nested loops and conditional blocks visually clear.
- **Modular Functions:** Create small, dedicated functions rather than long, mixed-purpose scripts (e.g., separate functions for Sobel gradients, A* algorithm, path smoothing, and Base64 encoding).
- **Use Comments and Docstrings:** Use # for short comments and triple-quote docstrings for describing function purpose, input parameters, and output values.
- **Handle Exceptions Gracefully:** Use try-except blocks while loading image models (.pth files), reading satellite data, or running API inference. Provide meaningful diagnostic messages.
- **Avoid Hardcoding:** Define constants such as DATA_DIR, BATCH_SIZE, LEARNING_RATE, and MODEL_SAVE_PATH to make the project flexible and easier to maintain.
- **Use Version Control:** Maintain the entire project in Git/GitHub with clean commit messages. Use .gitignore to exclude large datasets, model weights, and Python caches.

5.3 Testing Standards

To ensure the reliability, navigational accuracy, and operational readiness of the Lunar Command System, industry-recognized testing standards were followed. These include general software testing protocols as well as specialized practices applicable to machine learning and autonomous pathfinding systems.

- IEEE 829 — Standard for Software Test Documentation: Used to structure all test documentation, including test plans, test cases, and test results, ensuring traceability across testing activities.
- IEEE 1012 — Software Verification and Validation Standard: Guided both verification (checking the code matches mathematical pathfinding requirements) and validation (ensuring computed trajectories reflect safe, real-world rover scenarios).
- ISO/IEC/IEEE 29119 — Software Testing Standard: Used to define test strategies including functional testing, data validation checks, and performance testing of the pathfinding algorithm across varying terrain complexities.
- ISO/IEC 9126 / ISO/IEC 25010 — Software Quality Metrics: Used to evaluate software quality based on functionality, reliability, usability, efficiency, maintainability, and portability.
- Model-Specific Testing Practices: Cross-validation for U-Net segmentation stability; navigation-specific metrics (Fuel Cost, Safety Score, Path Distance, Hazard Intersections); sanity checks for data leakage; unit testing for tensor transformations and cost matrix fusion; and external testing using a separate, highly-cratered dataset.

Chapter 6: Conclusion and Future Scope

6.1 Conclusion

The development of the Lunar Command System for autonomous navigation has shown that fusing deep learning with physics-based algorithms can play a significant role in improving extraterrestrial pathfinding. By utilizing orbital satellite imagery, applying consistent tensor preprocessing, and evaluating architectures such as U-Net and SRResNet, the project demonstrates that automated models can accurately segment lunar terrain into Safe, Hazard, and Secondary conditions. The use of 3D surface rendering and Risk Heatmaps adds vital interpretability, enabling the visualization of the topographical regions influencing the A* routing predictions and enhancing operational trust. Overall, the project highlights the potential of combining neural networks with classical kinematics to support mission controllers by offering fast, consistent, and reliable navigational assistance, contributing positively to the field of autonomous space exploration and rover survivability.

6.2 Future Scope

Although the project establishes a strong foundation for visual-based lunar navigation, several areas offer opportunities for further improvement. Incorporating edge-computing capabilities directly onto rover hardware would enable the system to execute real-time path corrections without relying on Earth-bound server inference. Exploring more advanced architectures—such as Vision Transformers (ViTs) for terrain segmentation or Reinforcement Learning (RL) for dynamic obstacle avoidance—could further boost routing efficiency and robustness. Integrating multi-modal sensor telemetry, including live LiDAR scans or thermal imaging, may enhance decision-making and reduce false-positive hazard predictions caused by deep orbital shadows. Expanding the system to factor in dynamic rover mechanics (such as wheel slippage models, payload weight, or battery drain curves) rather than relying solely on static gradient costs would provide deeper operational value. Additionally, improving dataset diversity by training on other planetary bodies (such as Martian terrain) and validating the system across larger and more extreme simulated environments will be essential for real-world aerospace deployment.

References

- [1] Ronneberger, O., Fischer, P., and Brox, T. "U-Net: Convolutional Networks for Biomedical Image Segmentation." Proceedings of MICCAI, 2015.
- [2] Ledig, C., et al. "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network." IEEE CVPR, 2017.
- [3] Hart, P., Nilsson, N., and Raphael, B. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." IEEE Transactions on Systems Science and Cybernetics, 1968.
- [4] Maturana, D., and Scherer, S. "3D Convolutional Neural Networks for Landing Zone Detection from LiDAR." Proceedings of ICRA, 2015.
- [5] NASA Jet Propulsion Laboratory. "Mars Science Laboratory — Curiosity Rover Autonomous Navigation System." Technical Report, JPL-TRS, 2012.

Individual Contribution

Project Contribution and Findings

GAUTAM KUMAR (Machine Learning & Data Processing)

Primary responsibility in this project involved collecting, organizing, and preprocessing the Kaggle lunar surface datasets. Handled image formatting, prepared high-resolution reference pairs, and processed the semantic segmentation masks. Responsible for implementing, training, and evaluating the deep learning models: the SRResNet for super-resolution upscaling and the Custom U-Net for terrain hazard classification (Safe, Hazard, Secondary). Optimized training loops on GPU environments, tuned hyperparameters, and extracted the crucial Class 1 (Hazard) probability tensor to act as the AI Risk Map. This work significantly improved skills in PyTorch, computer vision, and building end-to-end deep learning pipelines.

SAYED SAJID ALI (Backend API & Algorithms)

Primary contribution focused on the backend architecture, computational physics, and pathfinding logic. Built the FastAPI server to handle the analyze_sector endpoints and process image payloads. Using OpenCV, implemented Sobel filters to calculate physical terrain gradients from the upscaled images. Mathematically fused these physical slopes with the AI risk map to generate a unified navigational Cost Matrix, and implemented the Advanced A* pathfinding algorithm with custom heuristics and a moving-average trajectory smoothing function. This work enhanced skills in Python, backend API routing, spatial algorithms, and mathematical modeling.

GITANSHU CHOUDHARY (Frontend Development & 3D Visualization)

Responsible for designing and developing the Mission Control HUD (Heads-Up Display) user interface. Using React and Vite, built a highly responsive frontend that communicates with the FastAPI backend via Axios. Primary technical challenge was transforming the backend's 2D terrain_z arrays into fully interactive 3D surface meshes using Plotly.js, projecting A* path coordinates onto this 3D map, and programming the "Initiate Drive" animation sequence. Also implemented dynamic Elevation Profile graphs, Sector Composition pie charts, and the Ground

Truth vs. AI output image toggles. This experience strengthened understanding of React state management and UI/UX design for complex aerospace applications.

Contribution to Project Report Preparation

- Gautam Kumar: Contributed to chapters covering Machine Learning Methodology, Dataset Description, Preprocessing Steps, and Model Architectures (SRResNet, U-Net). Prepared training pipeline explanations, AI segmentation performance metrics, and handled overall report formatting, proofreading, and consistency checks.
- Sayed Sajid Ali: Contributed to chapters detailing the Backend System Architecture, Physics Engine, and Pathfinding Algorithms. Prepared detailed explanations of the OpenCV Sobel slope calculations, Cost Matrix fusion equations, and evaluation of routing metrics.
- Gitanshu Choudhary: Contributed to chapters covering Implementation Details, User Interface Design, and System Interpretability. Prepared explanations of the 3D rendering logic (Plotly.js), API data flow, and provided HUD screenshots, heatmap visualizations, and elevation graphs.

Contribution to Project Presentation and Demonstration

- Gautam Kumar: Introduced the project, explained the problem statement regarding GPS-denied navigation, and detailed the deep learning architectures used. Demonstrated how the U-Net identifies hazards from raw orbital feeds and presented the AI training evaluation metrics.
- Sayed Sajid Ali: Explained the physics engine and pathfinding algorithms. Demonstrated how the AI output is merged with OpenCV slope calculations to form the Cost Matrix, and walked the audience through the logic of the Advanced A* algorithm.
- Gitanshu Choudhary: Conducted the live demonstration of the Mission Control interface, showed the API in action by scanning a sector, navigated the interactive 3D Plotly surface, executed the "Initiate Drive" animation, and explained the real-time data analytics.

Plagiarism Report

(This report is mandatory for all projects. Plagiarism must be below 25%)

[Attach Turnitin Plagiarism Report Here]